

김진아

Backend · Infra Engineer

twrp3301@gmail.com · github.com/catapillar0505 · [Notion 포트폴리오](#)

99년생(만 27세) · 서울여자대학교 소프트웨어융합학과 졸업 (2020.03 ~ 2026.02)

교육

새싹 청년취업사관학교 — AWS·AI MSA 웹서비스 개발 영등포 8기

2026.05 ~ 2026.11

멋쟁이사자처럼 — JAVA 백엔드 부트캠프 15기

2025.02 ~ 2025.08

한이음 ICT멘토링 (과학기술정보통신부 지원)

2023.04 ~ 2023.11

자격증

정보처리기사 2025.08

기술 스택

Backend Java · Spring Boot · Python · FastAPI

DB / Cache MySQL · Redis

AI Serving YOLO · ONNX Runtime

Cloud / Infra AWS · Terraform · Ansible · Kubernetes · Docker · Nginx

CI/CD GitHub Actions · Jenkins · ArgoCD

수상

멋쟁이사자처럼 백엔드 15기 파이널 최우수(1위) — MYCE	2025.08	학생창업 아이디어경진대회 우수상 — Recordy	2023.11
Programming GURU 해커톤 Android 대상 — 메추라기	2024.02	소프트웨어융합학과 졸업 프로젝트 은상 — Stainless	2023.02
배리어프리 앱 개발 컨테스트 우수상 — Stainless	2024.02	Programming GURU 해커톤 Python 장려상	2023.02
한이음 ICT멘토링 공모전 입선 — Keep Work	2023.12	창업 아이디어 경진대회 대상 — Recordy	2022.10

성장 과정 — Stainless에서 Micro-Lens까지

시각장애인 보조라는 같은 문제를 3년 동안 계속 풀면서, 앱에서 백엔드·인프라로 범위를 넓혀 왔습니다

2023 — 시각장애인을 앱으로 시작

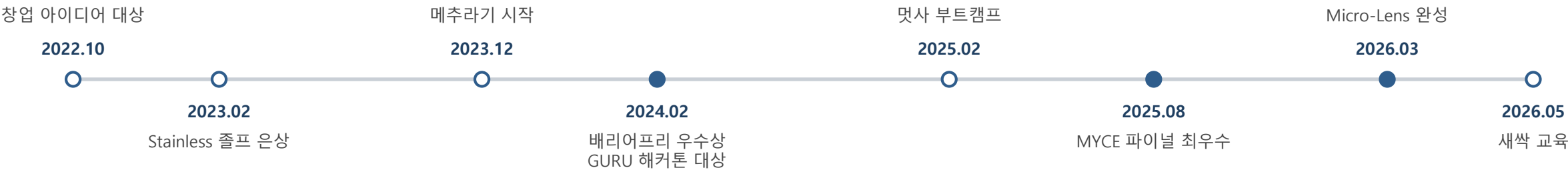
- 시각장애인 인터뷰로 필요한 기능을 확인하고
 웃 얼룩 탐지 앱 Stainless 개발
- 당시 서버·배포 역량이 없어 모델을 앱에 내장
 한 온디바이스 방식에 그침 — 모델 교체가 곧
 앱 재배포
- 배리어프리 앱 개발 컨테스트 우수상 · 졸업 프
 로젝트 은상

2023.12 ~ 2025 — 팀 프로젝트

- 메추라기: Terraform·Ansible IaC, 실시간 알림
 설계·재설계, 성능 병목 개선 (GURU 해커톤 대
 상)
- 멋쟁이사자처럼 부트캠프 → MYCE: 백엔드 설
 계·캐싱·팀 협업 (파이널 최우수 1위)
- 서버·인프라·배포처럼 온디바이스 때는 못 해
 본 것들을 실전에서 익힘

2026 — 혼자서 플랫폼으로 재구축

- Micro-Lens: Stainless를 K8s 서버 서빙 구조로
 다시 개발 (1인)
- 서버에서 모델만 바꿔 끼우면 되는 구조 — 얼
 룩·이물질에서 치아 검사까지 대상을 넓힘
- YOLOv12 재학습 mAP50 0.55 → 0.8, GitOps
 배포·HA까지 직접 운영



지금은 새싹 청년취업사관학교에서 AWS·MSA 과정을 들으며 다음 프로젝트를 준비하고 있습니다.

Stainless — 시각장애인용 옷 얼룩 탐지 앱

2023 · 팀 · 배리어프리 앱 개발 컨테스트 우수상 · 졸업 프로젝트 은상 | 담당: YOLOv5 학습 · TFLite 변환 · 온디바이스 추론 · 접근성

어떤 얼룩을 찍어도 '음료'로 분류 — 원인은 파싱 코드

TFLite 변환 후 어떤 얼룩도 음료로만 분류. 모델 재학습 대신 후처리를 의심해, 출력 텐서 파싱이 클래스 확률 인덱스를 5로 하드코딩(out[0][i][5])해 항상 0번 클래스만 읽던 버그를 찾아 수정. 박스당 1개 클래스만 반환하던 구조도 다중 클래스 탐지로 개선.

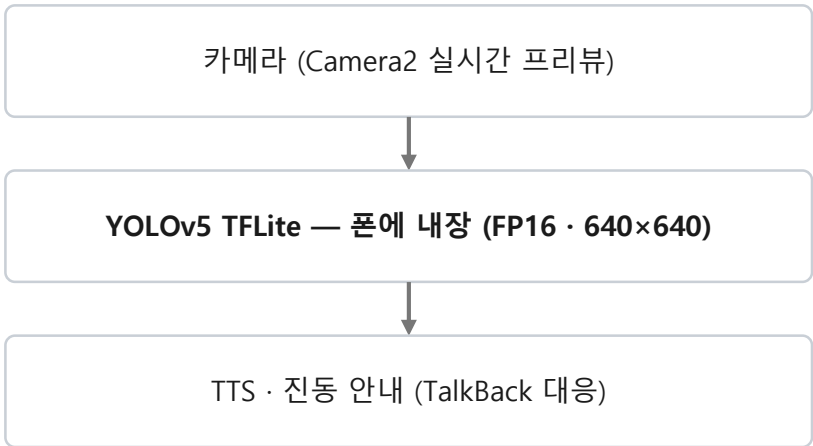
Camera2 캡처 — 사흘 막힌 끝에 롤백하고 다시 설계

정지사진 캡처를 추가하다 previewSize 불일치로 크래시 → 해상도 하드코딩 강제 통일 → 블랙스크린. 하 루치 작업을 롤백하고, 프리뷰 스트림은 그대로 두고 JPEG 캡처 경로를 분리하는 설계로 전환해 성공.

시각장애인 사용자에게 맞춘 조정

시각장애인에게는 오탐보다 미탐이 치명적 — confidence 0.6 → 0.3 하향으로 재현율 우선. TalkBack 대응은 단순 명칭이 아니라 "두번 탭하세요" 같은 조작 방법 포함 문구와 Q&A 전문 낭독까지 설계.

온디바이스 구조



당시의 한계 (→ Micro-Lens에서 해결)

- 모델 교체 = 앱 재배포
- 추론 성능이 기기에 의존
- mAP50 0.55 — 데이터 편향 (연한 얼룩 100/1,000장)
- 서버·인프라·배포 역량 부재

메추라기 ① 실시간 알림 — WebSocket+폴링에서 SSE로 재설계

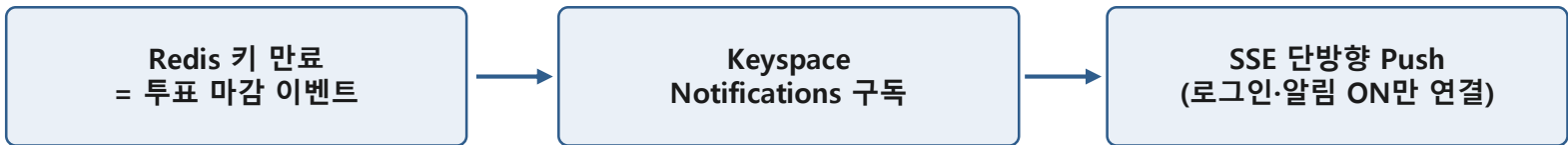
2023.12 ~ 2026.02 · 팀 · GURU 해커톤 대상 · AI 메뉴 추천·투표 커뮤니티 | 담당: IaC · 회원 인증 · 실시간 알림 · AI 추천 백엔드

AS-IS — 1차 구현 (WebSocket + 폴링)



지연 최대 60초 · 매분 DB 부하 · 알림에 불필요한 양방향 연결

TO-BE — 재설계 (SSE + Redis Keyspace Notifications)



실시간 · 이벤트 발생 시에만 동작 · Nginx proxy_buffering off 등 프록시 계층까지 설계

장애 대비 — 헬스체커(1분)가 Redis 이상 감지 시 DB 직접 조회 모드로 자동 전환하는 하이브리드 fallback

60초 → 0

알림 지연
(폴링 제거)

20개

1차 구현 시 작성한
단위·통합 테스트

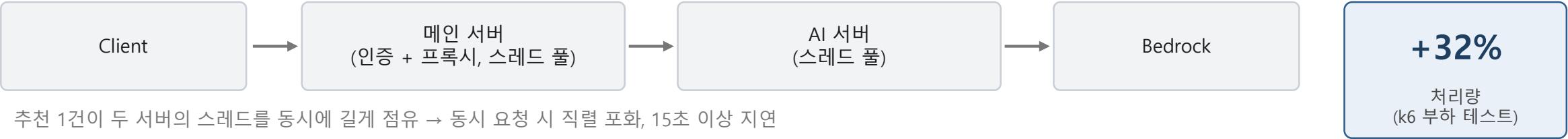
❗ 왜 다시 만들었나

1차 구현(WebSocket + 폴링, 테스트 20개 작성)을 운영해 보니 문제가 보였다 — 알림은 단방향이라 WebSocket까지 필요 없었고, 폴링은 t3.small(2GiB) 환경에 상시 부하였다. Redis Pub/Sub은 이벤트가 유실될 수 있어 DB 조회 fallback을 같이 설계했고, 전환 후 지연은 최대 60초에서 실시간이 됐다.

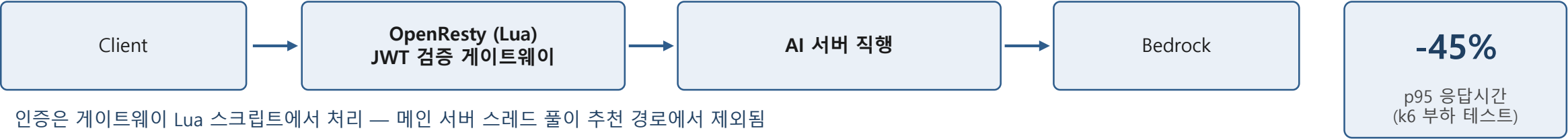
메추라기 ② AI 추천 15초 지연 — 병목 추적과 Direct Path

2023.12 ~ 2026.02 · 팀 · AI 메뉴 추천·투표 커뮤니티

Legacy 경로 — 두 서버 스레드 풀의 직렬 포화



Direct Path — 게이트웨이에서 인증만 하고 메인 서버 우회



병목 확인과 k6 검증

추천 요청이 15초 이상 걸리는 경로를 따라가며 메인 서버(인증.프록시)와 AI 서버 두 스레드 풀이 직렬로 포화되는 구조적 병목을 확인. 게이트웨이(OpenResty Lua)에서 JWT를 검증하고 AI 서버로 직행하는 Direct Path를 구성한 뒤, k6 부하 테스트로 legacy 대비 처리량 +32%, p95 응답시간 -45%를 검증.

Bedrock 호출 자체도 튜닝

불필요 응답 필드 제거 + max_tokens 4000 → 1000으로 비용·응답 속도 개선, 10분 시간 윈도우 캐시 키로 "다시 추천받기"가 같은 결과만 주던 문제 해결, 알레르기·기피 음식은 [절대 추천 금지] 프롬프트 제약으로 반영.

메추라기 ③ 인프라와 트러블슈팅

2023.12 ~ 2026.02 · 팀 · Terraform·Ansible AWS 프로비저닝 담당

| 코드를 안 바꿨는데 terraform plan이 서버를 지우려 했다

plan 결과 EC2 3대 destroy 예고. 원인은 콘솔에서 수동 생성한 CloudFront Function(코드 밖 리소스)과 most_recent AMI 갱신에 따른 인스턴스 replace 판정. terraform import로 콘솔 리소스를 코드화하고 lifecycle ignore_changes로 AMI 갱신을 차단해 'No changes' 상태 복구 — 과정을 트러블슈팅 문서로 남김.

| 카카오 로그인 500 에러 — Spring Security 필터 체인 내부까지 추적

가입된 이메일로 카카오 로그인 시 BusinessException이 OAuth2 필터 체인을 그대로 탈출해 FailureHandler가 호출되지 않고 500 반환. 예외 전파 경로 (Provider→ProviderManager→Filter)를 추적해 OAuth2AuthenticationException으로 래핑, 에러코드 기반 리다이렉트로 정상 동작하게 수정. JWT 필터의 콜백 경로 가로채기와 세션 정책(IF_REQUIRED)도 정리.

| S3 정적 배포의 동적 라우트 403 — 하루 8개 PR로 대안을 하나씩 검증

output:'export'는 서버 런타임이 없어 /community/5 같은 동적 라우트의 HTML이 없음 → 403. 전체 Client Component 전환, generateStaticParams, placeholder 생성 등 대안을 하루 만에 순차 검증한 끝에 쿼리 파라미터 방식으로 확정 — 새 데이터가 추가돼도 재빌드 불필요.

| AWS 구성은 전부 코드로

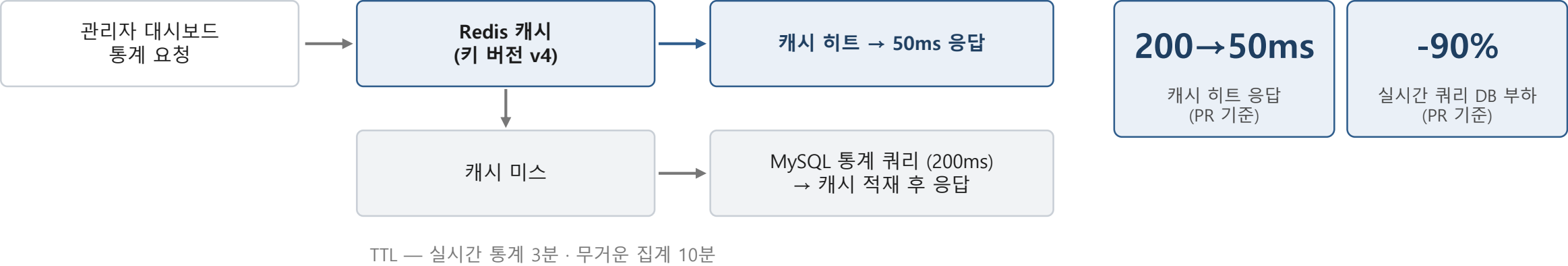
Terraform 모듈(EC2·VPC·SES·S3)과 Ansible role로 AWS 구성 전체를 프로비저닝, GitHub Actions 배포 자동화·CloudWatch 로깅. SES bounce 억제 설정, SSE용 Nginx 프록시 구성까지 인프라 계층을 담당.

기술 Java · Spring Boot · Spring Security · MySQL · Redis · SSE · AWS Bedrock · OpenResty(Lua) · Terraform · Ansible · GitHub Actions · Next.js · k6

[main](#) · [ai](#) · [infra](#) · [client](#) · [시연영상](#)

MYCE ① 실시간 통계 대시보드 — 하이브리드 캐싱

2025 · 팀 · 멧쟁이사자처럼 백엔드 15기 파이널 최우수(1위) · 박람회 예약·운영 관리 플랫폼 | 담당: 박람회 관리자 도메인 (백엔드·프론트)



지표 성격에 따라 캐시를 둘로 나눔

오늘 예약·시간대별 입장 같은 실시간 지표는 Redis TTL 캐시로, 누적·결제처럼 정확성이 중요한 지표는 RDB 직접 조회 + 별도 캐시로 나누는 2단계 하이브리드. Redis 복합 객체의 LinkedHashMap 역직렬화 오류는 타입 가드(불일치 시 캐시 미스 처리)로 방어.

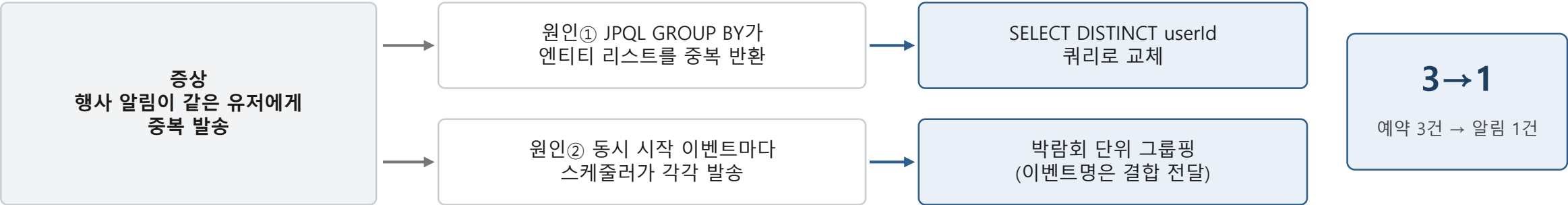
수치가 안 맞던 문제 — 세 번에 걸쳐 개선

예약·결제 테이블 간 수치 불일치를 데이터 소스 통일(Reservation 기준)로 해결하고, 집계 로직 변경 후 이전 캐시가 남는 문제는 캐시 키 버전 관리(v2→v4)로 배포 즉시 무효화. 운영 중 대응을 위해 refresh/clear를 분리한 캐시 삭제 API와 프론트 도구까지 — 공유 Redis라 전체 영향을 팀에 공지하고 진행.

MYCE ② 알림 중복 발송 디버깅과 QR 도메인 분리

2025 · 팀 · 박람회 관리자 도메인 담당

알림 중복 발송 디버깅



QR 도메인 관심사 분리 — 4개 서비스로



중복 코드 40행 → 0행 · 평균 클래스 크기 229행 → 58행 · Swagger 회귀 테스트 + 문서화

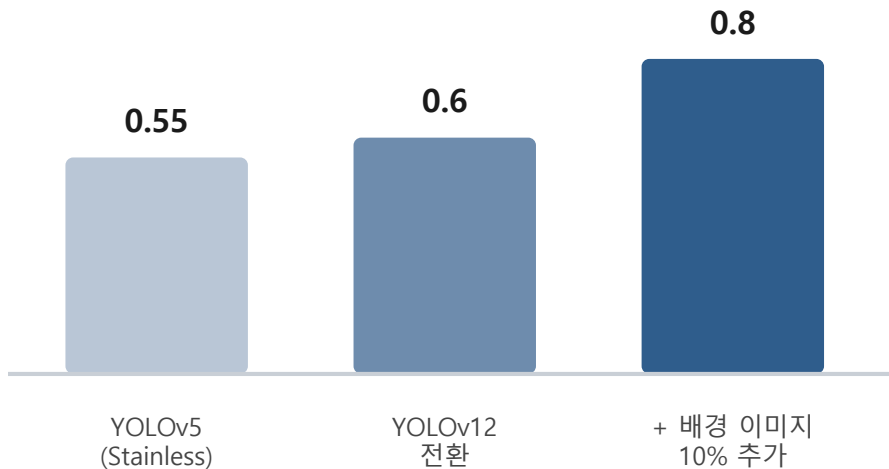
팀 컨벤션을 문서로 정리하고 코드에 적용

JPA 더티체킹 동작 원리와 Setter 없는 엔티티 설계(의도가 드러나는 도메인 메서드 패턴)를 저장소 이슈로 정리해 공유. 서비스 계층에서 권한 검증을 가장 먼저 수행하는 validateMyPermission 패턴과 계층별 책임 규칙을 세워 관리자 CRUD 전반에 적용. React 19 대시보드(Chart.js)·i18n 등 프론트 화면도 함께 구현.

Micro-Lens ① 데이터셋 분석으로 mAP50 0.55 → 0.8

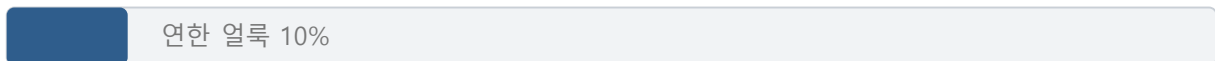
2026.03 · 1인 프로젝트 · 시각장애인용 얼룩·이물질 AI 탐지 웹 서비스 — Stainless를 플랫폼으로 재구축

mAP50 개선 추이

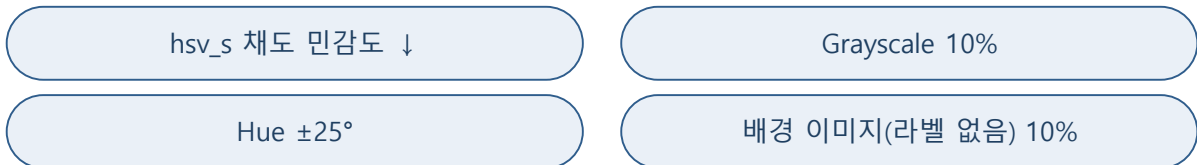


데이터셋의 문제 — 연한 얼룩 편향

전체 1,000장 중 연한 얼룩 100장 (10%) · 보라·파랑 계열 부족



대응 — 색보다 패턴을 학습하도록



모델 구조도 분석 — 국소 특징에 집중하는 CNN이 전체 맥락에 약하다고 보고 attention 기반 YOLOv12로 전환

모델 검증과 학습-서빙 전처리 불일치 수정

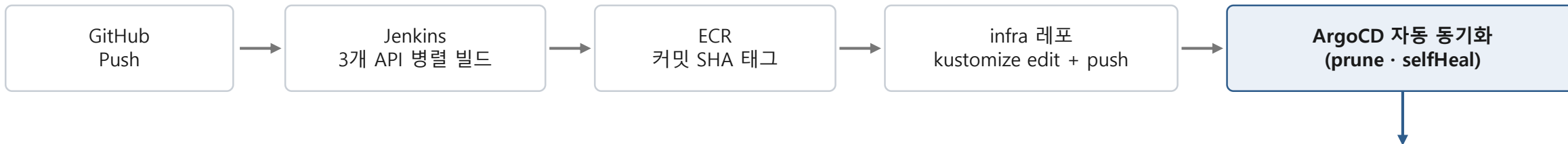
train/val loss 추이를 관찰해 과적합 여부를 검증하며 mAP50 0.8 달성. 서빙 단계에서는 학습(Ultralytics-OpenCV)과 추론 서버(자작 PIL letterbox)의 전처리 불일치(train-serve skew)를 발견하고 동일한 OpenCV 구현으로 교체.

재학습 데이터가 쌓이는 구조

사용자가 업로드한 얼룩 데이터를 서버에 축적해 재학습에 활용하는 구조 — 모델 가중치를 S3 버전 관리 + init container로 배포와 분리했기에 재학습 모델을 앱 재배포 없이 교체 가능.

Micro-Lens ② kubeadm K8s 직접 구축과 GitOps 배포

2026.03 · 1인 프로젝트 · Terraform 모듈 + Ansible 플레이북 10여 개로 서버 구성 전 과정 자동화



수동 배포가 selfHeal과 충돌 — 배포를 Git으로 통일

Ansible+Jinja2 수동 kubectl 배포는 ArgoCD selfHeal이 수동 변경을 계속 되돌려 충돌. Kustomize base/overlay(phase1 검증 → phase2 전체+HA)로 전환해 Git을 단일 소스로 확립.

EKS 없이 직접 해결한 것들

kubeadm 클러스터는 ECR 자동 인증이 없어 kubelet credential provider(IAM Role + IMDSv2, 12h 캐시)를 직접 구성. NAT 인스턴스는 iptables가 eth0 하드코딩으로 무효(Nitro는 ens5)인 문제를 잡고, 맥등 Ansible + systemd oneshot으로 재부팅에도 유지.

GPU 전환 대비 설계

gpu_enabled 변수 몇 개만 바꾸면 NVIDIA 드라이버·Time-Slicing 플레이북으로 GPU 환경이 재구성되도록 설계.

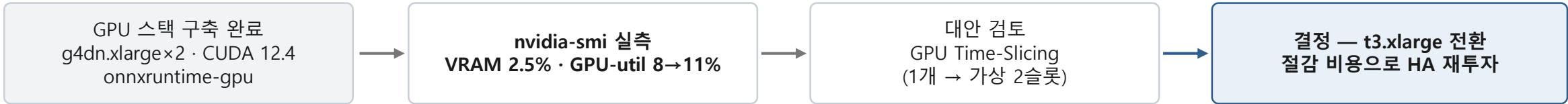
K8s 클러스터 — kubeadm 직접 구축

- 마스터 1 + 워커 3 (EC2 6대 중, 프라이빗 서브넷) · Calico CNI
- 추론 파드 replicas 2 + podAntiAffinity 노드 분산 → 노드 장애에도 서비스 유지
- 모델 가중치: S3 버전 관리 → init container 다운로드 (배포 주기 분리)
- Route 53 → ALB(TLS 종료) → Nginx Ingress

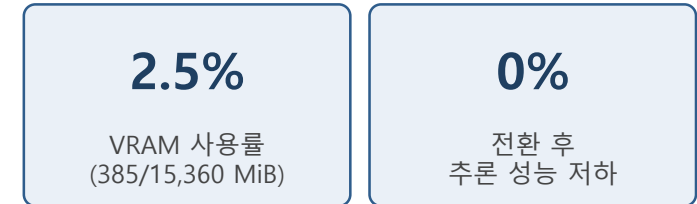
Terraform 모듈: vpc · ec2 · s3 · ecr · route53 · acm · alb (S3 원격 상태 + DynamoDB 락)
| Ansible: NAT → containerd-kubeadm → Calico → Jenkins → ArgoCD → 모델 업로드

Micro-Lens ③ GPU 실측 후 CPU 전환 — 절감 비용은 HA에

2026.03 · 1인 프로젝트



실측 근거 — T4 VRAM 사용률



실측해 보니 GPU가 거의 놀고 있었다

운영 중 nvidia-smi 관측 — VRAM 385MiB/15,360MiB(2.5%), 추론 요청에도 GPU-util 8→11% 변화뿐. Time-Slicing(노드당 파드 2개)까지 검토했지만 근본적으로 GPU가 불필요하다고 판단, CPUExecutionProvider로 고정하고 베이스 이미지도 CUDA에서 python:3.10-slim으로 경량화. 성능 저하 없이 비용 절감.

절감 비용으로 HA 구성 — 롤링업데이트 데드락 해결

절감분으로 추론 파드 replicas 2 + podAntiAffinity 분산 배치를 구성해 노드 1대 장애에도 서비스 유지. GPU 시절엔 노드당 GPU 1개가 전부 점유돼 기본 RollingUpdate(maxSurge 25%)의 새 파드가 영원히 Pending에 빠지는 데드락이 있었는데, maxSurge 0 / maxUnavailable 1로 순서를 바꿔 해결.

기술 AWS · Terraform · Ansible · Kubernetes(kubeadm) · ArgoCD · Jenkins · Docker · Nginx · FastAPI · ONNX Runtime · YOLOv12

[infra](#) · [ai-api](#) · [client](#) · [시연영상](#)